



Programming Language Concepts

Compiling Micro-SML

Niels Hallenberg Peter Sestoft



- Grammar for Micro-SML
- Example programs
- Using the compiler and byte code machine
- Compiler phases
- Evaluation of micro-SML
- Exceptions



File	Contents
MicroSml/Absyn.fs	the abstract syntax
MicroSml/FunLex.fsl	lexer specification
MicroSml/FunPar.fsy	parser specification
MicroSml/TypeInference.fs	type inference algorithm
MicroSml/HigherFun.fs	interpreter
MicroSml/Contcomp.fs	backward compiler with local optimizations
MicroSml/ParseTypeAndRun.fs	combining lexer, parser, interpreter and compiler
MicroSml/Machine.fs	micro-SML stack machine instruction definitions
MicroSml/MicroSMLC.fs	read arguments from command line
MicroVM/microvm.c	micro-VM stack machine in C
SmlEx/*.sml	example programs for micro-SML



Grammar	Comment
<i>Prog:</i> <code>vds begin e end</code>	A program has global declarations and an expression.
<i>Valdecs:</i> <code>fun f x = e</code> <code>and g x = e</code> <code>val x = e</code> <code>exception exn</code>	Mutually recursive functions; the first one. Additional functions. Value declaration, global or let-bound local. Exception declaration, global or let-bound local.
<i>Expr:</i> <code>i</code> <code>x</code> <code>true</code> <code>false</code> <code>nil</code> <code>if e then e else e</code> <code>fn x -> e</code>	Integer constant. Variable access. Boolean constant true. Boolean constant false. Empty list constant. Conditional expression. Anonymous function.



Grammar	Comment
<code>let <i>vds</i> in <i>e</i> end</code>	Local declarations and an expression.
<code><i>e</i> <i>e</i></code>	Function application
<code>raise <i>e</i></code>	Exceptions can be raised.
<code>try <i>e</i> with <i>exn</i> -> <i>e</i> end</code>	Exceptions can be handled.
<code>print <i>e</i></code>	A built-in print primitive.
 <code><i>e</i> + <i>e</i></code>	Binary operators such as +, -, *.
<code><i>e</i> && <i>e</i></code>	Logical operators such as && and .
 <code><i>e</i> :: <i>e</i></code>	List construction.
<code>isnil <i>e</i></code>	Test for empty list.
<code>hd <i>e</i></code>	Head of list.
<code>tl <i>e</i></code>	Tail of list.
 <code><i>e</i> ; <i>e</i></code>	Sequence of two expressions.

Example program `ex01.sml`



`f` and `g` are mutually recursive functions.
`print` adjusts to the type of value to print.

```
fun f x = x + g 4  
and g x = x
```

```
begin  
  print (f 1)  
end
```

Executing using compiler and byte code machine:

```
% dotnet run ex01.sml  
Micro-SML compiler v 1.1 of 2018-11-18  
Compiling ex01.sml to ex01.out
```

```
Compiled to file ex01.out  
% ./Msm1VM/src/msmlmachine ex01.out  
5  
Result value: 5  
Used 0 cpu milli-seconds
```



`genList` generates a list of integers. `map` maps `f` on each element in the list.

```
fun genList n =  
  let  
    fun loop n = fn acc ->  
      if n < 0 then acc else loop (n-1) (n::acc)  
  in  
    loop n nil  
  end  
  
fun map f = fn xs ->  
  if isnil xs then nil else f (hd xs) :: map f (tl xs)  
  
begin  
  let  
    val xs = genList 10  
  in  
    print xs;  
    print (map (fn x -> x + 1) xs)  
  end  
end
```



```
% dotnet run ex02.sml
Micro-SML compiler v 1.1 of 2018-11-18
Compiling ex02.sml to ex02.out

Compiled to file ex02.out
% ./Msm1VM/src/msmlmachine ex02.out
[0,1,2,3,4,5,6,7,8,9,10][1,2,3,4,5,6,7,8,9,10,11]
Result value: 2684371612
Used 0 cpu milli-seconds
```




See the `README` for instructions to compile the compiler:

```
dotnet build
```

```
dotnet run SmlEx/ex01.sml
```

The compiler takes four optional arguments:

`-opt` will enable simple optimizations similar to micro-C.

`verbose` will output intermediate AST with type information and byte code instructions.

`-eval` will execute the program with the interpreter as part of this compilation phase.

`-debug` will print various debug information.

```
% dotnet run -opt test01.sml
```

```
Micro-SML compiler v 2.0 of 2026-05-30
```

```
Compiling SmlEx/ex02.sml to SmlEx/ex02.out.
```



See the `README` for instructions to compile the byte code machine.

The byte code machine `microvm` takes two optional arguments:

`-trace` will output the stack content at each instruction to execute.

`-silent` will suppress output from the garbage collector.

The machine does not have a garbage collector implemented !!!

You can add your garbage collector to the machine. It should work the same as in your assignment.

```
% ../MicroVM/microvm SmlEx/test01.out
```

```
true true true true true ...
```

```
Result value: 1
```

```
Used 0 cpu milli-seconds
```

```
Number of GC: 0
```



The compiler consists of the following phases:

- Lexing
- Parsing
- Tailcall analysis (result AST output, if option `-verbose`)
- Type inference (result AST output, if option `-verbose`)
- Evaluate program (with option `-eval`)
- Compilation (with optimizations if option `-opt`)



The AST can hold additional information 'a option.

```
type expr<'a,'b> =  
  | CstI of int * 'a option  
  | CstB of bool * 'a option  
  | CstN of 'a option  
  | Var of string * 'a option  
  | AndAlso of expr<'a,'b> * expr<'a,'b> * 'a option  
  | OrElse of expr<'a,'b> * expr<'a,'b> * 'a option  
  | Seq of expr<'a,'b> * expr<'a,'b> * 'a option  
  | Prim2 of string * expr<'a,'b> * expr<'a,'b> * 'a option  
  | Prim1 of string * expr<'a,'b> * 'a option  
  | If of expr<'a,'b> * expr<'a,'b> * expr<'a,'b>  
  | Fun of string * expr<'a,'b> * 'a option  
  | Call of expr<'a,'b> * expr<'a,'b> * bool option * 'a option  
  | Let of valdec<'a,'b> list * expr<'a,'b>  
  | Raise of expr<'a,'b> * 'a option  
  | TryWith of expr<'a,'b> * exnvar * expr<'a,'b>  
  ...
```



```
type expr<'a> =  
  ...  
and valdec<'a,'b> =  
  | Fundecs of (string * string * expr<'a,'b> * 'b option) list  
  | Valdec of string * expr<'a,'b> * 'b option  
  | Exndec of exnvar  
and exnvar =  
  | ExnVar of string  
and program<'a,'b> =  
  | Prog of valdec<'a,'b> list * expr<'a,'b>
```

A value declaration `valdec<'a, 'b>` is either a group of mutually recursive functions, variable binding or an exception.



Expression e	Status of subexpressions
let $x = e_r$ in e_b end	e_b is in tail position, e_r is not
$e_1 + e_2$	neither e_1 nor e_2 is in tail position
if e_1 then e_2 else e_3	e_2 and e_3 are in tail position, e_1 is not
let $f\ x = e_r$ in e_b end	e_b is in tail position, e_r is not
$f\ e$	e is not in tail position
$e_1 ; e_2$	e_1 is not in tail position
fn $x \rightarrow e$	
fun $f\ x = e$	
and $g\ x = e$	
val $x = e$	e not tail position as result must be stored



`tc'` is called with `true` for constructs in tail position.

```
let tailcalls p : program<'a,'b> =  
  let rec tc' tPos e =  
    match e with  
    | CstI _ -> e  
    ...  
    | Seq(e1,e2,aOpt) -> Seq(tc' false e1,tc' tPos e2,aOpt)  
    | Fun(x,e,aOpt) -> Fun(x,tc' true e,aOpt)  
    | Call(e1,e2,_,aOpt) ->  
      Call(tc' false e1,tc' false e2,Some tPos,aOpt)  
    | Let(valdecs,letBody) ->  
      Let(List.map (tcValdec' false) valdecs,  
          tc' tPos letBody)  
  and tcValdec' tPos = function  
    | Valdec(x,eRhs,bOpt) -> Valdec(x,tc' tPos eRhs,bOpt)  
    | Fundecs(fs) ->  
      Fundecs(List.map (fun (f,x,e,bOpt) ->  
                          (f,x,tc' true e,bOpt)) fs)  
  and tcProg' = function  
    | Prog(valdecs,body) ->  
      Prog(List.map (tcValdec' false) valdecs,tc' true body)  
  tcProg' p
```



Compiling `ex02.sml` with options and verbose enabled we get the program pretty printed with tail calls enabled:

```
% dotnet run -opt -verbose ex02.sml
```

Zooming in on the `genList` function

```
fun genList n =  
  let  
    fun loop n = fn acc ->  
      if n < 0 then acc else loop (n-1) (n::acc)  
    in loop n nil end
```

we get:

Program with tailcalls:

```
fun genList n =  
  let  
    fun loop n =  
      fn acc -> if (n < 0) then acc  
                else loop (n - 1)_tail (n :: acc)  
    in loop n_tail nil end
```




A variable occurrence x is bound if it is in e_2 of a binding

`let val  $x = e_1$  in  $e_2$  end.`

Otherwise it is free.

The same for functions

`let fun  $f$   $x_1 = e_1$  in  $e_2$`

or

`fn  $x_2 \rightarrow e_3$`

f is bound in e_1 and e_2 . x_1 is bound in e_1 .

x_2 is bound in e_3 .

Which occurrences are **bound** and which are **free**:

`let val  $z=x$  in  $z+x$  end`

`let`

`fun  $f$   $x = x+3$`

`and  $g$   $x = f$   $x$`

`in`

`$f$  1 +  $g$   $x$`

`end`

`fn  $x \rightarrow x + 1 + y$`



Free variables are used to build closures.

Subtract bound variables from the free variables in value and function declarations.

```
let rec freevars e : string Set =  
  match e with  
  | Let(valdecs, letBody) ->  
    let (fvs, bvs) =  
      List.fold freevarsValdec (Set.empty, Set.empty) valdecs  
      (freevars letBody) + fvs - bvs  
  | Fun(x, fBody, _) -> freevars fBody - (set [x])  
  | ...  
and freevarsValdec (fvs, bvs) = function  
  Valdec(x, eRhs) ->  
    (fvs + ((freevars eRhs) - set [x]), bvs + set [x])  
  ...
```

Free variables in `fvs` and bound variables in `bvs`.

Free variables example (ex10.sml)



```
val n = 42
begin
  let
    val x = 1                (* fvs=[]      bvs=[x]          *)
    val y = x + 1            (* fvs=[x]     bvs=[x,y]       *)
    val z = x + y            (* fvs=[x,y]  bvs=[x,y,z]    *)
    fun f x = x + y + n      (* fvs=[n,y]  bvs=[x,y,z,f]  *)
    val g = fn x -> z + (f x) (* fvs=[f,z]  bvs=[x,y,z,f,g] *)
  in
    x+y
  end
end
```

Try compile with `-debug` option to see the variable environment and free variables in the closure for `f`.

```
% dotnet run -debug SmlEx/ex10.sml
```

```
...
```

```
DEBUG Fundecs: f, parameter x:
```

```
  fvsAll: Freevars = [ n,y ]
```

```
  fvsGlobalsInScope: Freevars = [ n ]
```

```
  fvs in clos: Freevars = [ y ]
```

```
  varEnv: fDepth = 4
```

```
  [ f -> Locvar[3]
```

```
  z -> Locvar[2]
```

```
...
```



free and **bound** variables marked below for function `f` and the `let`-expression.

Notice the two declarations of `y`.

```
begin
  let
    val y = 1                (* fvs1=[]   bvs1=[y]   *)
    fun f x =                (* fvs2=[y]   bvs1=[y,f]  *)
      let
        val z = y + 1       (* fvs2=[y]   bvs2=[z]   *)
        val y = 1           (* fvs2=[]   bvs2=[z,y] *)
      in
        z+y+x
      end
    in
      f y
    end
  end
end
```

The algorithm *freevars* can't see the difference between the two declarations of `y` and concludes that there are NO free variables to the function body of `f`.

Free Variables – BUG (ex11.sml)



```
% dotnet run -debug SmlEx/ex11.sml
Micro-SML compiler v 2.0 of 2026-05-30
Compiling SmlEx/ex11.sml to SmlEx/ex11.out.
```

...

```
DEBUG Fundecs: f, parameter x:
  fvsAll: Freevars = [  ]
  fvsGlobalsInScope: Freevars = [  ]
  fvs in clos: Freevars = [  ]
  varEnv: fDepth = 2
[ f |-> Locvar[1]
y |-> Locvar[0]
__exnNum__ |-> Glovar[0] ]
```

...

```
Comp.lookup: y not found
```

The occurrence of y in $\text{val } z = y + 1$ is not in the environment when compiling the body of f .

The two occurrences of y are two different variables and should be treated as such by the compiler.



Consider the micro-SML program `ex12.sml`. The two declarations of `x` makes a “hole” in the scope of the first declaration of `x`.

```
begin
  let val x = 2 in
    let val x = 5 in
      x + x
    end
  end
end
```

```
begin
  let val x1 = 2 in
    let val x2 = 5 in
      x2 + x2
    end
  end
end
```

Renaming variables such that all variable declarations are unique is called *alpha conversion*.

The exercise 13.4 is to implement alpha conversion which will solve the problem of calculating free variables (`ex11.sml`, `ex12.sml`).



$$\frac{}{\rho \vdash \text{nil} : t \text{ list}} (g1)$$

$$\frac{\rho \vdash e : t \text{ list}}{\rho \vdash \text{hd } e : t} (g2)$$

$$\frac{\rho \vdash e : t \text{ list}}{\rho \vdash \text{tl } e : t \text{ list}} (g3)$$

$$\frac{\rho \vdash e : t \text{ list}}{\rho \vdash \text{isnil } e : \text{bool}} (g4)$$

$$\frac{\rho \vdash e_1 : t \quad \rho \vdash e_2 : t \text{ list}}{\rho \vdash e_1 :: e_2 : t \text{ list}} (g5)$$

$$\frac{\rho \vdash e : \text{exn}}{\rho \vdash \text{raise } e : t} (g6)$$

Rule *g1*: `nil` must fit any type of list.

Rule *g6*: A `raise` expression must match the type of the expression it appears in.



$$\frac{\rho \vdash e_1 : t \quad \rho \vdash e_2 : t \quad \rho(\text{exn}) = \text{exn}}{\rho \vdash \text{try } e_1 \text{ with } \text{exn} \rightarrow e_2 : t} \quad (g7)$$

$$\frac{\rho \vdash e_1 : t_1 \quad \rho \vdash e_2 : t_2}{\rho \vdash e_1 ; e_2 : t_2} \quad (g8)$$

$$\frac{\rho[x \mapsto t_x] \vdash e : t_r}{\rho \vdash \text{fn } x \rightarrow e : t_x \rightarrow t_r} \quad (g9)$$

$$\frac{\rho \vdash e_r : t_r \quad \rho[x \mapsto \forall \alpha_1 \dots \alpha_n. t_r] \vdash e_b : t \quad \alpha_1 \dots \alpha_n \text{ not free in } \rho}{\rho \vdash \text{let val } x = e_r \text{ in } e_b \text{ end} : t} \quad (g10)$$

$$\frac{\rho[\text{exn} \mapsto \text{exn}] \vdash e_b : t}{\rho \vdash \text{let exception } \text{exn} \text{ in } e_b \text{ end} : t} \quad (g11)$$

Rule *g7*: like an `if` expression.

Rule *g11*: Type `exn` is used for exceptions.



$$\begin{array}{c}
 \rho' = \rho[f_1 \mapsto t_{x1} \rightarrow t_{r1}, \dots, f_n \mapsto t_{xn} \rightarrow t_{rn}] \\
 \rho'[x_i \mapsto t_{xi}] \vdash e_i : t_{ri} \quad i \in [1, \dots, n] \\
 \rho[f_1 \mapsto \forall \alpha_{11} \dots \alpha_{1m}. t_{x1} \rightarrow t_{r1}, \dots, f_n \mapsto \forall \alpha_{n1} \dots \alpha_{no}. t_{xn} \rightarrow t_{rn}] \vdash e_b : t \\
 \hline
 \alpha_{11} \dots \alpha_{no} \text{ not free in } \rho \quad (g12) \\
 \text{let fun } f_1 \ x_1 = e_1 \\
 \rho \vdash \quad \vdots \\
 \text{and } f_n \ x_n = e_n \quad \text{in } e_b \text{ end} : t
 \end{array}$$

```

fun h x = x + g (23::nil) (* x has type int *)
and g x = hd x           (* x has type int list *)
    
```

The types for x in the two functions g and h differ and the type rule would not work if all type parameters were extended to ρ' when typing e_i .



We need a type for exceptions and lists.

```
type typ =  
  ...  
  | TypL of typ (* list types *)  
  | TypeE      (* exception type *)
```

Unification is extended with the new types.

```
let rec unify t1 t2 : unit =  
  let t1' = normType t1  
  let t2' = normType t2  
  match (t1', t2') with  
  ...  
  | (TypL t1, TypL t2) -> unify t1 t2  
  | (TypeE, TypeE) -> ()
```

The example `ex05.sml` annotated with type information:

Program with types:

```
fun h x = (x:int + g:((int list) -> int)  
          (23:int :: nil:(int list)):(int list):int):int  
and g x = hd(x:(int list)):int
```



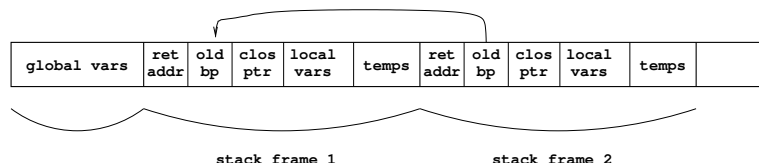
- `NIL`, `CONS` to handle lists.
- `PUSHLAB`, `HEAPSTI`, `HEAPLDI`, `ACLOS` to create and use closures on the heap.
- `CLOSCALL` and `TCLOSCALL` to call closure allocated functions.

Instruction	Stack before	Stack after	Effect
1 <code>NIL</code>	s	$\Rightarrow s, \text{nil}$	Push <code>nil</code> on stack
42 <code>HEAPCOPY</code> n	$s, v_1, \dots, v_n, p$	$\Rightarrow s, p$	Copy from stack to heap
43 <code>HEAPLDI</code> n	s, p	$\Rightarrow s, v$	Push n 'th value of closure p
44 <code>HEAPALLOC</code> <i>tag</i> n	s	$\Rightarrow s, p$	Allocate closure on heap
73 <code>PUSHLAB</code> / <code>PUSHLAB</code>	s	$\Rightarrow s, l$	Push label l
82 <code>CLOSCALL</code> m	$s, cp, v_1, \dots, v_m$	$\Rightarrow s, r, bp, cp, v_1, \dots, v_m$	Call function closure cp
83 <code>TCLOSCALL</code> m	$s, r, bp, cp_1,$ $u_1, \dots, u_n,$ $cp_2, v_1, \dots, v_m$	$\Rightarrow s, r, bp, cp_2, v_1, \dots, v_m$	Tail call to closure cp_2
90 <code>PRINTI</code>	s, v	$\Rightarrow s, v$	Print integer v
92 <code>PRINTB</code>	s, v	$\Rightarrow s, v$	Print boolean v
96 <code>PRINTL</code>	s, v	$\Rightarrow s, v$	Print list v
110 <code>CONS</code>	s, v, t	$\Rightarrow s, p$	Create cons cell on heap

Stack frame with closure pointer



Stack frame now contains a *closure pointer* pointing at the closure for the called function containing free variables.



The compile time variable environment has been extended with closure stored variables and temporary variables declared when computing globals (`Contcomp.fs`):

```
type var =  
  | Glovar of int           // Absolute address on stack  
  | Locvar of int          // Address relative to base pointer  
  | Closvar of int         // Address relative to closure  
  | GloTmpvar of int       // Absolute address on stack
```



The `print` instruction chosen depends on the annotated type on the AST.

`CAR` and `CDR` is used to access the first and second component of a cons cell.

$E[\text{nil}] = \text{NIL}$

$E[\text{print } e] =$

$E[e]$

PRINTX

where e has type integer, boolean or list

PRINTI , PRINTB or PRINTL depending on type

$E[\text{hd } e] = E[e]$

CAR

$E[\text{tl } e] = E[e]$

CDR

$E[\text{isnil } e] =$

$E[e]$

NIL

EQ



A[Glovar addr] = **addr is an absolute address.**

CSTI addr
LDI

A[Locvar offset] = **offset is relative to current bp.**

GETBP
CSTI offset
ADD
LDI

A[Closvar offset] = **offset is relative to current cp.**

GETBP
LDI **put cp on stack.**
HEAPLDI offset **load value relative to cp.**

A free variable is fetched from the closure.

GloTmpvar is similar to **Glovar**.

How do we avoid CSTI and ADD instructions when accessing closure stored variables?



The free variables $fv_1, \dots, fv_n$ are calculated as described earlier. The code label *lab1* is also copied into the closure.

$VD[\text{fun } f \ x = e] =$	
ACLOS <i>sizeClos</i>	Allocate closure
PUSHLAB <i>lab1</i>	
A[fv_1]	Fetch free variables
...	to go in the
A[fv_n]	closure
A[f]	Fetch closure pointer
HEAPSTI <i>sizeClos</i>	Move data to closure
INCSP -1	Pop closure pointer
$F[\text{fun } f \ x = e] =$	
E[e]	Compile body
RET 2	Two arguments: closure and argument



$VD[\text{val } x = e] =$
 $E[e]$

$VD[\text{exception } \text{exn}] =$	<code>nextExnNumCode</code>
<code>CSTI addr</code>	Address of global exception counter
<code>CSTI addr</code>	Address of global exception counter
<code>LDI</code>	Load exception number
<code>CSTI 1</code>	
<code>ADD</code>	Increase number by one
<code>STI</code>	Store new number and leave on stack.

Above implements the generative nature of exceptions in micro-SML.



$E[\text{let } vds \text{ in } e \text{ end}] =$	where vds are value declarations
$VD[vds]$	Code for the vds .
GETSP	
CST $numVals - 1$	
SUB	Address of first vds on stack.
$E[e]$	Compile e .
STI	Store result of e at first vds on stack.
INCSP $-numVals$	Pop vds from stack.

$E[e1 \ e2] =$	
$E[e1]$	
$E[e2]$	Always one argument.
CLOSCALL 1	Use TCLOSCALL 1 in case of tail call.

The vds in the let construct must be popped after evaluating the body e and still keep the result of e .

- 1 Calculate the address of the first value declaration: GETSP, CST $numVals - 1$ and SUB.
- 2 After evaluating e move the result of e in the slot occupied by the first value declaration (STI).
- 3 Pop the remaining values (INCSP $-numVals$).



Exceptions are generative, meaning each declared exception is unique.

The example `exn04.sml` illustrates the generative behaviour:

```
fun genExn x = let exception E in E end
fun genRaise E = fn x -> raise E
fun genTryWith E = fn g -> try g 1 with E -> 1
begin
  let
    val E1 = genExn 1
    val h1 = genTryWith E1
    val E2 = genExn 2
    val r2 = genRaise E2
  in
    h1 r2
  end
end
```

The same exception `E` is declared but two different exception values are returned, hence `E1` and `E2` do not match.

The call `h1 r2` will not match `E` in the `try-with` expression and an unhandled exception is the result of the evaluation.



$$\frac{\rho \vdash e \Rightarrow \text{exn}}{\rho \vdash \text{raise } e \Rightarrow \text{exn}} \quad (h7)$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad v_1 \text{ is not an exception value } \text{exn}}{\rho \vdash \text{try } e_1 \text{ with } \text{exn} \rightarrow e_2 \Rightarrow v_1} \quad (h8)$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad v_1 \text{ is an exception value } \text{exn}', \text{ and } \text{exn} = \text{exn}' \quad \rho \vdash e_2 \Rightarrow v_2}{\rho \vdash \text{try } e_1 \text{ with } \text{exn} \rightarrow e_2 \Rightarrow v_2} \quad (h8')$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash \text{raise } \text{exn}' \Rightarrow v_2}{\rho \vdash \text{try } e_1 \text{ with } \text{exn} \rightarrow e_2 \Rightarrow v_2} \quad (h8'')$$

$$\frac{\text{exn} = \text{freshExnName()} \quad \rho[x \mapsto \text{exn}] \vdash e_b \Rightarrow v_b}{\rho \vdash \text{let exception } x \text{ in } e_b \text{ end} \Rightarrow v_b} \quad (h11)$$



An expression e must be evaluated to an exception value before it can be thrown.

For `try-with`, the exception value exn is put on the stack before the exception handler is pushed `PUSHHDLR`.

```
E[[raise e]] =  
    E[[e]]  
    THROW  
  
E[[try e1 with exn -> e2 end]] =  
    A[[exn]]  
    PUSHHDLR lab1  
    E[[e1]]  
    POPHDLR  
    GOTO lab2  
lab1: E[[e2]]  
lab2:
```



Instruction	Stack before	After	Effect
120 THROW	$s_1, \text{exn}_1, a, h, s_2, \text{exn}_2$	$\Rightarrow s_1$	Handler found $\text{exn}_1 = \text{exn}_2$; go to a
120 THROW	s, exn_2	$\Rightarrow _$	No handler found; abort
121 PUSHDLR a	s, exn	$\Rightarrow s, \text{exn}, a, hr$	Push handler
122 POPDLR	s, exn, a, h, v	$\Rightarrow s, v$	Pop handler; preserve v

- PUSHDLR pushes the exception and handler address on the stack together with *handler register* hr ; hr is updated to point at this exception handler on the stack.
- POPDLR pops the handler and resets hr .
- THROW searches the stack of handlers and jumps to the exception code if a matching handler is found.

Draw example handler list on the stack as an exercise.

See also Section 11.6.